Name:	Student ID:

ENTROPY, INFORMATION AND OPTIMAL TRANSPORT PROBLEM SHEET

This sheet covers the lectures on entropy, information theory and optimal transport. Question 1 practises entropy and code lengths, Question 2 mutual information, the KL divergence and cross-entropy, and Question 3 the maximum entropy method. Each ends with a short Python part (`import numpy as np` is assumed). Question 4 is harder and optional: you write a small optimal transport solver in PyTorch and use it to move points by gradient descent. Logarithms are base 2 (bits) unless we say otherwise.

1

(a) Find the entropy of (i) a fair coin, (ii) a coin with $\mathbb{P}(\text{head}) = 0.9$, and (iii) a random variable X taking the values a, b, c, d with probabilities $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{8}$. [4 marks]

$\frac{}{4}$

(b) We code the outcomes of X in part (a)(iii) as $a \to 0$, $b \to 10$, $c \to 110$, $d \to 111$. Why can a string of these code words always be decoded? Decode `0110100111`. Find the expected code length and compare it with the entropy. [4 marks]

$\frac{}{4}$

(c) Instead we use two bits for every outcome (`00`, `01`, `10`, `11`). How many extra bits per outcome does this cost? What is the largest entropy any distribution on four outcomes can have, and which distribution has it? [4 marks]

4

(d) X_1 and X_2 are independent copies of X. What is the entropy of the pair (X_1, X_2)? [3 marks]

3

(e) Write a Python function `entropy(p)` that returns the entropy in bits of a probability vector (treat $0\log 0$ as 0). Check part (a), then draw 10000 random distributions on four outcomes with `rng.dirichlet(np.ones(4))` and find the largest entropy. [5 marks]

5

End of question 1

(a) $\frac{}{4}$	(b) $\frac{}{4}$	(c) $\frac{}{4}$	(d) $\frac{}{3}$	(e) $\frac{}{5}$	Total $\frac{}{20}$

2

(a) Bits X and Y have the joint distribution

	$Y=0$	$Y=1$
$X=0$	0.4	0.1
$X=1$	0.1	0.4

Find H_X, H_Y, $H_{X,Y}$, $H_{X|Y}$ and the mutual information $I_{X;Y}$. [5 marks]

5

(b) Explain why this table describes sending equally likely bits down a binary symmetric channel. What is the error rate f, and what is the channel capacity $1 - H(f)$? [3 marks]

3

(c) Let p be a fair coin and q a coin with $\mathbb{P}(\text{head}) = 0.9$. Find $\mathrm{KL}(p\|q)$ and $\mathrm{KL}(q\|p)$ in bits. What does $\mathrm{KL}(p\|q)$ mean in terms of code lengths? [4 marks]

4

(d) For some input, the true label distribution over three classes is $p = (0.7, 0.2, 0.1)$ and a classifier predicts $q = (0.6, 0.3, 0.1)$. Using natural logarithms, find the expected cross-entropy loss $\mathbb{E}_{Y\sim p}\big[-\ln q(Y)\big]$, the entropy H_p and $\mathrm{KL}(p\|q)$, and check that the loss is $H_p + \mathrm{KL}(p\|q)$. [4 marks]

$\frac{\quad}{4}$

(e) In Python, check part (d), then show numerically that the KL divergence is never negative by computing it for 10000 random pairs of distributions on five outcomes.

[4 marks]

$\frac{\quad}{4}$

End of question 2

(a) $\frac{\quad}{5}$	(b) $\frac{\quad}{3}$	(c) $\frac{\quad}{4}$	(d) $\frac{\quad}{4}$	(e) $\frac{\quad}{4}$	Total $\frac{\quad}{20}$

3 Maximum entropy with discrete variables needs only ordinary Lagrange multipliers. We use natural logarithms here, so $H = -\sum_i p_i \ln p_i$.

(a) A dice has faces $i = 1,\ldots,6$ with probabilities p_i. Maximise the entropy subject only to $\sum_i p_i = 1$, using the Lagrangian $\mathcal{L} = -\sum_i p_i \ln p_i + \lambda_0\big(\sum_i p_i - 1\big)$. What distribution do you get? [4 marks]

4

(b) Now we are told only that the average score is μ: $\sum_i i p_i = \mu$. Add a multiplier β for this constraint and show that

$$p_i = \frac{\mathrm{e}^{\beta i}}{Z(\beta)}, \qquad Z(\beta) = \sum_{j=1}^{6} \mathrm{e}^{\beta j}.$$

[5 marks]

5

(c) Which value of β gives $\mu = 3.5$? Should β be positive or negative if $\mu = 4.5$? Explain. [3 marks]

3

(d) In Python, find the β that gives $\mu = 4.5$ (use `scipy.optimize.brentq` on $\mu(\beta) - 4.5$, or bisection), and print the probabilities and the entropy in bits. [5 marks]

5

(e) Why is this a sensible guess for the dice, and when might it be a poor one? [3 marks]

3

End of question 3

(a) $\frac{}{4}$	(b) $\frac{}{5}$	(c) $\frac{}{3}$	(d) $\frac{}{5}$	(e) $\frac{}{3}$	Total $\frac{}{20}$

4 This question is harder and optional. We move a probability mass p_i at points x_i onto a mass q_j at points y_j, where moving a unit of mass from x_i to y_j costs d_{ij}. A transport plan $\gamma_{ij} \geq 0$ has row sums $\sum_j \gamma_{ij} = p_i$ and column sums $\sum_i \gamma_{ij} = q_j$, and the optimal transport cost is the smallest $\sum_{i,j} d_{ij}\gamma_{ij}$.

(a) In one dimension, with $d_{ij} = |x_i - y_j|$ and equal masses, the optimal plan matches the points in sorted order. Find the cost of moving equal masses $\frac{1}{3}$ at $x = (0,1,3)$ to $y = (2,4,5)$. Is the optimal plan unique? [4 marks]

$\overline{4}$

(b) The Sinkhorn algorithm sets $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$, starts from $\boldsymbol{u} = \mathbf{1}$, and repeats $\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^{\mathsf{T}}\boldsymbol{u})$, $\boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K}\boldsymbol{v})$, where $\oslash$ is element-by-element division. The plan is $\gamma_{ij} = u_i K_{ij} v_j$. Show that straight after the $\boldsymbol{v}$ update the column sums of γ are exactly q_j, and straight after the $\boldsymbol{u}$ update the row sums are exactly p_i. [4 marks]

$\overline{4}$

(c) Implement Sinkhorn in PyTorch for part (a) (start with `torch.set_default_dtype(torch.float64)`: $\mathrm{e}^{-d/\epsilon}$ is tiny). Run 1000 iterations for $\epsilon = 1$, 0.1 and 0.01, and print the cost $\sum_{ij} d_{ij}\gamma_{ij}$ and the row and column sums. Look at the plan for $\epsilon = 0.01$. [7 marks]

7

(d) Because every step is a PyTorch operation, the Sinkhorn cost can be differentiated. Put your code in a function `sinkhorn_cost(x, y, eps=0.1, iters=200)` for points in two dimensions, with equal masses and squared distances $d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{y}_j\|^2$. Take 30 target points $\boldsymbol{y}_j$ equally spaced on the circle of radius 1 about $(3,0)$, start 30 points $\boldsymbol{x}_i$ at random (`torch.randn(30, 2, requires_grad=True)`), and run 300 steps of `torch.optim.SGD([x], lr=0.5)` on the Sinkhorn cost. Where do the points end up? Why squared distances? [10 marks]

$\overline{10}$

End of question 4

(a) $\overline{\quad}$ (b) $\overline{4}$ (c) $\overline{7}$ (d) $\overline{10}$ Total $\overline{25}$